Web-programozás 1 – Gyakorlat Beadandó feladat

Forma-1 weboldal dokumentációja

Készítette:

Harmatiné Pásztor Éva (JBDAQO)

Deák Dávid (OPFHRV)

Projekt eléri útvonala és belépéshez szükséges adatok:

- GitHub projekt URL cím: https://github.com/ttdd33/Webprog-Gyak
- Weboldal URL cím: http://f1oldal.nhely.hu/
- Internetes tárhely és adatbázis belépéséhez szükséges FTP és URL cím:
  - Internetes tárhely: https://www.nethely.hu/
  - Email: pasztor.eva91@gmail.com
  - Jelszó: 4VC!L3m29ftQPzX
  - FTP kiszolgáló: ftp.nethely.hu
  - Felhasználó: pasztor.eva91
  - Jelszó: 4VC!L3m29ftQPzX

Tartalomjegyzék

Bevezetés 3

1. A projekt előkészítése 4

2. Alaprendszer beállítása és felhasználói felület kialakítása 5

3. Reszponzív működés kialakítása és hibajavítás 7

4. HTML5 szemantikus elemek alkalmazása 10

5. Képgaléria és képfeltöltés megvalósítása 11

6. CRUD menü megvalósítása 13

7. Főoldal megvalósítása 18

8. Kapcsolat menü megvalósítása 19

9. Üzenetek menü megvalósítása 20

10. Rövid használati útmutató a honlaphoz 21

11. Összegzés 22

12. Források 24

Bevezetés

Jelen dokumentáció célja a Webprogramozás I. gyakorlat beadandó feladat keretében elkészített webalkalmazás részletes bemutatása. A feladat során egy komplex, több menüpontból álló weboldalt kellett létrehozni PHP, HTML, CSS és JavaScript technológiák felhasználásával, amely adatbázis-kezelést, felhasználókezelést, képfeltöltést, valamint CRUD műveleteket is tartalmaz.

A projektet kétfős csoportban valósítottuk meg, amely során külön figyelmet fordítottunk a feladatok összehangolására, a munkafolyamatok megtervezésére, valamint a verziókövetés tudatos alkalmazására. A közös fejlesztés során folyamatos kommunikációt tartottunk fenn, és a feladatokat egymással egyeztetve, egymást segítve valósítottuk meg. A projekt során a GitHub verziókezelő rendszert használtuk, amely lehetővé tette a fejlesztési folyamat átlátható követését, a módosítások dokumentálását, valamint a részfeladatok elkülönítését. A verziókezelés során törekedtünk arra, hogy ne egyetlen lépésben töltsük fel a kész projektet, hanem több, egymást követő fejlesztési állapotban rögzítsük a változtatásokat, megfelelve ezzel a feladat követelményeinek.

A fejlesztés során a kiadott feladatleírásban előírt front-controller tervezési mintát alkalmaztuk, amely egy központi vezérlőn keresztül irányítja az alkalmazás működését. Ez a megoldás biztosította a projekt egységes felépítését, valamint lehetővé tette, hogy az egyes funkciókat (például képfeltöltés, CRUD műveletek, űrlapkezelés) jól elkülöníthető és karbantartható módon valósítsuk meg.

A projekt során a feladatkiírásban szereplő összes kötelező elemet megvalósítottuk. Ennek keretében elkészült:

- a reszponzív, több szintű navigációs menü,
- a felhasználókezelés (regisztráció, bejelentkezés, kijelentkezés),
- a főoldal multimédiás tartalommal (videók, térkép),
- a képgaléria és képfeltöltési funkció,
- a kapcsolat űrlap kliens- és szerveroldali ellenőrzéssel,
- az üzenetek adatbázisban történő tárolása és megjelenítése,

- valamint egy teljes értékű CRUD alkalmazás egy kiválasztott adatbázis-tábla kezelésére.

A fejlesztés során külön figyelmet fordítottunk arra, hogy az alkalmazás ne csak technikailag feleljen meg a követelményeknek, hanem felhasználói szempontból is átlátható, logikusan felépített és esztétikus legyen. Ennek érdekében egységes arculatot alakítottunk ki, amely minden oldalon következetesen megjelenik, valamint biztosítottuk a mobil eszközökön való megfelelő működést is.

A projekt végső állapotában az alkalmazást sikeresen feltöltöttük internetes tárhelyre, ahol minden funkció működőképesen elérhető. Az adatbázis kapcsolatot a tárhely környezetéhez igazítottuk, és ellenőriztük a rendszer működését valós környezetben is.

A dokumentáció további részeiben részletesen bemutatjuk az egyes funkciók megvalósítását, a fejlesztés során felmerült problémákat, valamint azok megoldását, kiegészítve képernyőképekkel és magyarázatokkal, hogy pontosan nyomon követhető legyen, hogy a feladat egyes követelményeit hogyan és hol teljesítettük az alkalmazásban.

1. A projekt előkészítése

Első lépésként a kiadott feladatleírás alapján áttekintettük a követelményeket, és meghatároztuk, hogy milyen funkciókat kell megvalósítani az alkalmazásban. Kiemelt figyelmet fordítottunk arra, hogy minden kötelező feladatot teljesítsünk, mivel ezek hiányában a projekt nem értékelhető. A követelmények között szerepelt többek között a front-controller minta alkalmazása, a felhasználókezelés, a képfeltöltés, az űrlapkezelés, valamint egy CRUD rendszer kialakítása.

A projekt témájának meghatározásakor a rendelkezésre álló adatbázisok közül választottunk egyet (2-07-Forma-1-3 táblás), amelyhez a weboldal tartalmát és struktúráját igazítottuk. A választás során fontos szempont volt, hogy az adatbázis megfelelően alkalmas legyen a CRUD műveletek bemutatására, valamint könnyen bővíthető legyen további funkciókkal. Ennek megfelelően egy Forma–1 témájú adatbázist használtunk, amely pilóták adatait tartalmazza (név, nem, születési dátum, nemzet), és jól illeszkedik a feladat követelményeihez.

A fejlesztéshez szükséges környezetet helyi gépen alakítottuk ki. A szerveroldali működéshez XAMPP környezetet használtunk, amely biztosította a PHP futtatását és a MySQL adatbázis kezelését. Az adatbázis kezelésére a phpMyAdmin felületet alkalmaztuk, ahol létrehoztuk a szükséges adatbázisokat és táblákat, valamint elvégeztük az adatok importálását is.

A fejlesztés során a Visual Studio Code szerkesztőt használtuk, amely megfelelő támogatást nyújtott a PHP, HTML, CSS és JavaScript nyelvekhez. A projekt fájlszerkezetét tudatosan alakítottuk ki annak érdekében, hogy az alkalmazás átlátható és jól karbantartható legyen. A front-controller minta alkalmazásával a központi vezérlést egyetlen belépési pont (index.php) biztosította, amely a különböző oldalak betöltéséért felelős.

A projekt struktúrája a következő főbb mappákra tagolódik:

- **includes/** – adatbázis kapcsolat és konfigurációs fájlok
- **logicals/** – az alkalmazás logikai működéséhez tartozó PHP fájlok (CRUD, belépés, regisztráció stb.)
- **templates/pages/** – az egyes oldalak megjelenítéséért felelős sablonok
- **styles/** – CSS fájlok az oldal kinézetének kialakításához
- **images/ és kepek/** – statikus képek és feltöltött képek tárolására

A közös munka során a GitHub verziókezelő rendszert használtuk. A projektet egy közös repository-ban tároltuk, amelyhez mindkét csoporttag hozzáfért. A fejlesztés során törekedtünk arra, hogy a módosításokat több lépésben, jól elkülöníthető commitok formájában rögzítsük. Ez nemcsak a feladat követelményeinek felelt meg, hanem megkönnyítette a hibák visszakövetését és a fejlesztési folyamat áttekintését is.

A projekt előkészítése során tehát sikerült egy stabil alapot kialakítani, amelyre a későbbi fejlesztési lépések biztonságosan és átlátható módon épülhettek. Ez az alapos tervezés nagyban hozzájárult ahhoz, hogy a projekt végül minden követelménynek megfelelően, működőképes formában készüljön el.

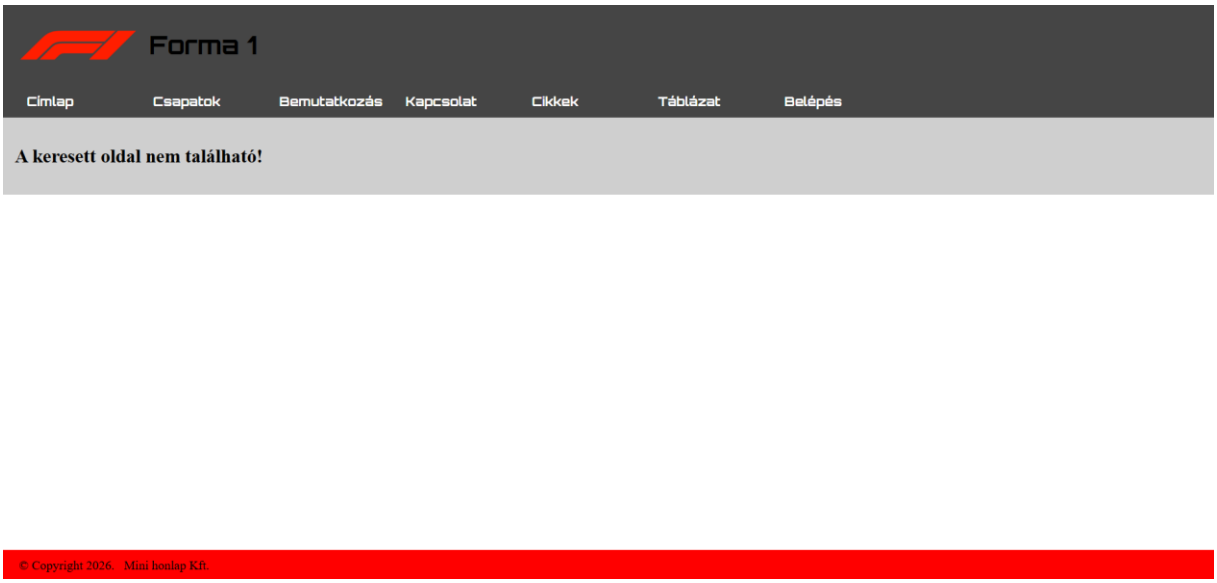
2. Alaprendszer beállítása és felhasználói felület kialakítása

A projekt megvalósítását a kiadott mintaalkalmazás bemásolásával kezdtük, amely a front-controller tervezési mintát alkalmazza. A kezdeti tesztelés során több hibát is észleltünk, elsősorban az oldalak közötti navigációval kapcsolatban.

A problémát az okozta, hogy egyes hivatkozások nem megfelelően voltak kialakítva, ezért a rendszer olyan URL-eket próbált megnyitni, amelyek a fejlesztői környezetben nem léteztek. Ezt a hibát az útvonalak javításával oldottuk meg, több esetben a „?” karakter beillesztésével, amely lehetővé tette a GET paraméterek helyes kezelését. Ennek eredményeként a front-controller működése helyreállt, és az oldalak közötti navigáció megfelelően működött.

A hibák javítását követően elindítottuk a fejlesztői környezetet az XAMPP segítségével. Az Apache és MySQL szolgáltatások elindítása után a phpMyAdmin felületen keresztül létrehoztuk az alkalmazáshoz szükséges adatbázist, majd importáltuk a mellékelt SQL fájlt. Az alapfunkciók ellenőrzése során megállapítottuk, hogy a regisztráció és a bejelentkezés megfelelően működik.

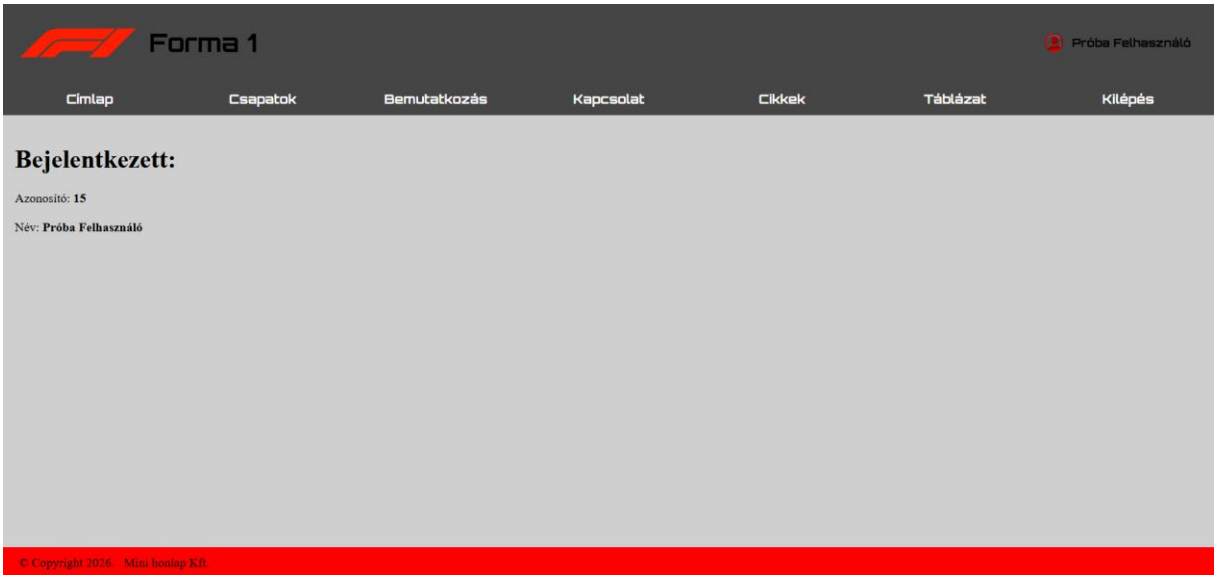
Ezt követően a felhasználói felület kialakítására helyeztük a hangsúlyt. A navigációs menüt átalakítottuk úgy, hogy az megfeleljen a feladat követelményeinek: vízszintes elrendezést alkalmaztunk, valamint több szintű menüszerkezetet hoztunk létre. A menü működését a felhasználó bejelentkezési állapotához igazítottuk, így a „Belépés” és „Kilépés” menüpontok nem jelennek meg egyszerre, hanem mindig az aktuális állapotnak megfelelő opció látható.



A felhasználói élmény javítása érdekében megvalósítottuk azt is, hogy bejelentkezett felhasználó esetén a fejléc jobb felső sarkában egy ikon és a felhasználó neve jelenjen meg, amely egyértelmű visszajelzést ad az aktuális állapotról.

Az oldal megjelenésének egységesítése érdekében a láblécet úgy állítottuk be, hogy az minden esetben a képernyő alján jelenjen meg, még akkor is, ha az adott oldal tartalma nem tölti ki a teljes rendelkezésre álló területet. Emellett a menü szélességét a teljes képernyőhöz igazítottuk, minimális szélesség megadásával, hogy kisebb felbontás esetén se torzuljon az elrendezés.

A tartalmi részek kialakításánál törekedtünk az egységes megjelenésre, ezért a háttérszínt és az elrendezést minden oldalon azonos módon állítottuk be. Ennek eredményeként egy átlátható, egységes és felhasználóbarát felületet sikerült kialakítani, amely megfelelő alapot biztosított a további funkciók megvalósításához.



3. Reszponzív működés kialakítása és hibajavítás

A projekt során kiemelt figyelmet fordítottunk arra, hogy az alkalmazás különböző képernyőméreteken, különösen mobil eszközökön is megfelelően működjön. Ennek érdekében a reszponzív megjelenést CSS módosításokkal és meglévő minták kombinálásával valósítottuk meg.

A fejlesztés során a navigációs menü átalakítása után azt tapasztaltuk, hogy kisebb képernyőméret esetén a lenyíló almenük nem működnek megfelelően. A probléma oka az volt, hogy a CSS-ben az almenük megjelenítése display: block értékre volt állítva, így azok folyamatosan láthatóak maradtak, és nem viselkedtek lenyíló menüként.

A hiba javítása érdekében módosítottuk a stílusbeállításokat, és az almenük alapértelmezett megjelenítését display: none értékre állítottuk. Ennek köszönhetően az almenük rejtett állapotból indulnak, és csak felhasználói interakció hatására jelennek meg, ami megfelel a kívánt működésnek.

A reszponzív működés kialakítása során több különböző forrásból származó megoldást kombináltunk. A főmenü mobilnézetben történő megjelenítéséhez a Bootstrap keretrendszer beépített navigációs megoldását alkalmaztuk, amely a „hamburger menü” működést biztosítja. Ez a megoldás a navbar, navbar-toggler és collapse navbar-collapse elemek segítségével valósul meg (oldal-07.html).

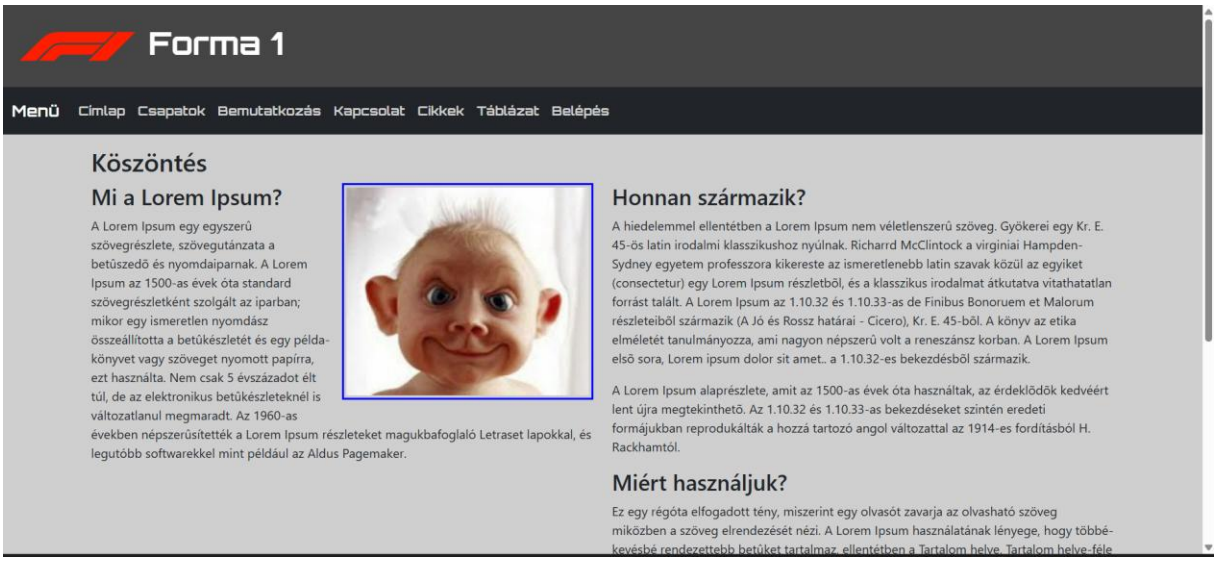
Az almenük kezeléséhez egy többszintű menü mintát használtunk, amelyben a második és harmadik szint külön osztályokkal (second-level-menu, third-level-menu) van definiálva (forrása: a tobbszintu-menu2.html). Asztali nézetben ezek az almenük alapértelmezetten rejtettek (display: none) és abszolút pozicionálással jelennek meg, míg mobil nézetben a megjelenítésüket módosítottuk.

A mobilbarát működés érdekében az alábbi kiegészítő beállításokat alkalmaztuk:

- position: static – hogy az almenük ne lebegjenek ki az oldalról,
- width: 100% – hogy teljes szélességben jelenjenek meg,
- margin-left beállítás – a jobb olvashatóság érdekében.

Ezek a módosítások biztosítják, hogy mobil nézetben az almenük ne oldalra nyíljanak ki, hanem egymás alatt, jól áttekinthető módon jelenjenek meg.

A tartalom elrendezésénél rácsszerkezetet (grid) alkalmaztunk, amely lehetővé teszi, hogy nagyobb képernyőn két oszlopban jelenjen meg a tartalom, míg kisebb képernyőn az elemek egymás alá kerüljenek. Ez hozzájárul a jobb olvashatósághoz és a felhasználóbarát megjelenéshez.



A fejlécben található „navbar-brand” elemet is módosítottuk annak érdekében, hogy dinamikusan az oldal aktuális címét jelenítse meg. Ennek megvalósításához a következő megoldást alkalmaztuk:

```
<a class="navbar-brand" href="."><?= $fejlec['cim'] ?></a>
```

Ez a módosítás biztosítja, hogy a fejléc minden oldalon az aktuális tartalomhoz igazodó címet jelenítse meg, ami javítja a felhasználói élményt és az alkalmazás átláthatóságát.

A fenti módosítások eredményeként sikerült egy olyan reszponzív felületet kialakítani, amely különböző eszközökön is megfelelően működik, és egységes felhasználói élményt biztosít.

4. HTML5 szemantikus elemek alkalmazása

A projekt fejlesztése során törekedtünk a modern HTML5 szabványok alkalmazására, különös tekintettel a szemantikus elemek használatára. Ennek célja az volt, hogy a weboldal szerkezete átláthatóbbá, logikusabbá és szabványosabbá váljon.

A tartalmi részek strukturálásához a korábban használt egyszerű megoldások helyett szemantikus elemeket alkalmaztunk. Ennek keretében az egyes tartalmi blokkokat <section> elemekkel tagoltuk, amely segíti a dokumentum logikai felépítésének egyértelműbb meghatározását.

Például a korábban önállóan használt címsor:

```
<h2>Köszöntés</h2>
```

helyett az alábbi, strukturáltabb megoldást alkalmaztuk:

```
<section>
  <h2>Köszöntés</h2>
</section>
```

Ez a megközelítés nemcsak a kód olvashatóságát javítja, hanem hozzájárul a szemantikus felépítéshez is, amely a későbbi bővítés és karbantartás során is előnyös.

A fejlesztés során a fő tartalmi terület szerkezetét is egyszerűsítettük. A korábbi megoldásban egy felesleges elem került beágyazásra a <main> elembe, amely nem adott hozzá valódi funkcionális értéket, ugyanakkor bonyolította a kódot.

A korábbi szerkezet:

```
<main id="wrapper">
  <div id="content">
    <?php include("./templates/pages/{$keres['fajl']}.tpl.php"); ?>
  </div>
</main>
```

átalakításra került egy egyszerűbb, letisztultabb formára:

```
<main id="content">
  <?php include("./templates/pages/{$keres['fajl']}.tpl.php"); ?>
</main>
```

Ezzel a módosítással megszüntettük a felesleges beágyazást, így a kód átláthatóbbá és könnyebben kezelhetővé vált.

Összességében a szemantikus elemek alkalmazásával sikerült egy modernebb, strukturáltabb és szabványosabb HTML felépítést kialakítani, amely hozzájárul az alkalmazás minőségéhez és hosszú távú fenntarthatóságához.

5. Képgaléria és képfeltöltés megvalósítása

A projekt egyik fontos funkciója a képgaléria és a képfeltöltés megvalósítása volt. A feladat követelményei alapján biztosítani kellett, hogy a feltöltött képeket minden felhasználó megtekinthesse, azonban új képet kizárólag bejelentkezett felhasználó tölthessen fel.

A megvalósításhoz a gyakorlati anyagban található PHP alapú képfeltöltési és galéria mintát használtuk fel, amelyet a saját projektünk struktúrájához igazítottunk. A képek tárolására a projekt gyökérkönyvtárában létrehoztunk egy külön **„kepek”** nevű mappát, amely a feltöltött fájlok fizikai tárolásáért felel.

A képek kezeléséhez több alapvető konfigurációs változót használtunk, amelyek meghatározzák a feltöltés feltételeit, például az engedélyezett fájltípusokat, a maximális fájlméretet, valamint a tárolási helyet. Ezek a beállítások biztosítják, hogy a rendszer csak megfelelő formátumú és méretű képeket fogadjon el.

A galéria működésének alapja, hogy a rendszer beolvassa a megadott mappában található fájlokat, majd csak azokat jeleníti meg, amelyek megfelelnek az engedélyezett kiterjesztéseknek (például .jpg, .png). A fájlok feldolgozása során a rendszer ellenőrzi, hogy valóban fájlokról van-e szó, majd azok módosítási idejét is figyelembe veszi. Ennek segítségével a képeket időrendben tudjuk megjeleníteni, így a legfrissebb feltöltések kerülnek előre.

A képfeltöltés megvalósítása során a Megoldasok\11-PHP - képek, galéria, képfeltöltés-Web-alkalmazások biztonsága\Képek, galéria, képfeltöltés minta fájlokat alkalmaztuk. A feltöltési folyamat során több ellenőrzést is beépítettünk:

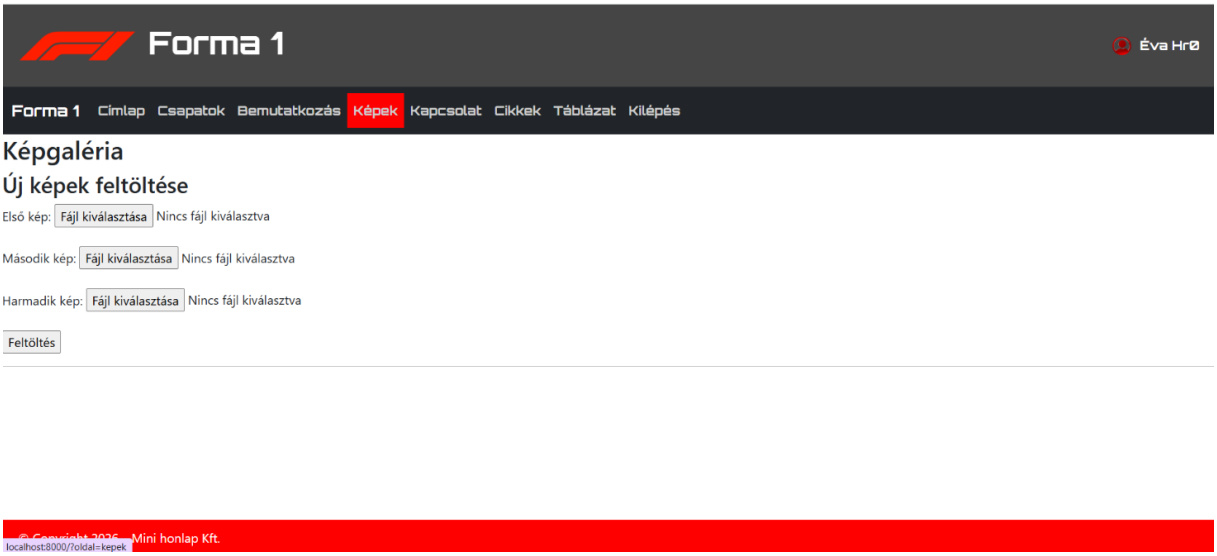
- ellenőriztük a fájl típusát, hogy csak képfájlok kerülhessenek feltöltésre,
- korlátoztuk a maximális fájlméretet,
- vizsgáltuk, hogy a fájl már létezik-e a célmappában,
- valamint kezeltük az esetleges feltöltési hibákat.

Sikeres feltöltés esetén a fájl a megadott könyvtárba kerül áthelyezésre, és azonnal megjelenik a galériában.

A funkció megvalósítása során fontos szerepet kapott a jogosultságkezelés is. A képfeltöltő űrlapot úgy alakítottuk ki, hogy az csak bejelentkezett felhasználók számára legyen elérhető. Ezzel biztosítottuk, hogy a rendszer megfeleljen a feladat követelményeinek, miszerint a képeket mindenki láthatja, de feltölteni csak az arra jogosult felhasználók tudnak.

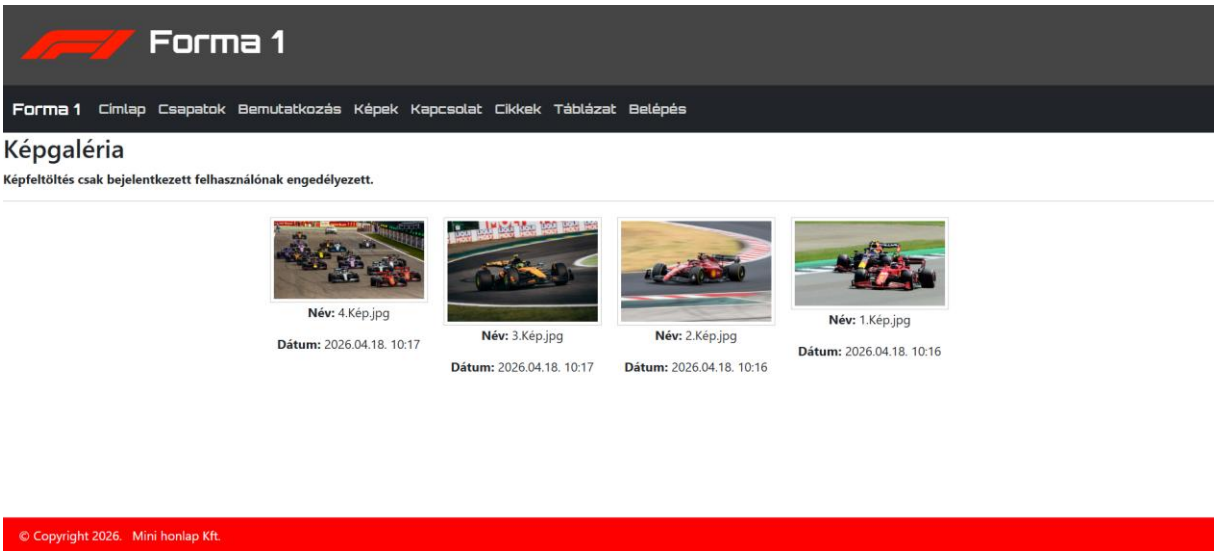
A fejlesztés során több hibával is találkoztunk. Az egyik probléma abból adódott, hogy a kódban egy megjegyzés nem lett megfelelően lezárva, ami miatt a program nem futott le. A hiba javítása után a rendszer működőképessé vált.

Egy másik hiba során a feltöltött képek nem jelentek meg az oldalon, csak egy üres Képgaléria volt látható. Ennek oka az volt, hogy a képek tárolására létrehozott mappa neve nem egyezett meg a kódban megadott elérési úttal (például „Képek” helyett „kepek”). A mappanév javítása után a galéria megfelelően működött, és a képek megjelentek az oldalon.

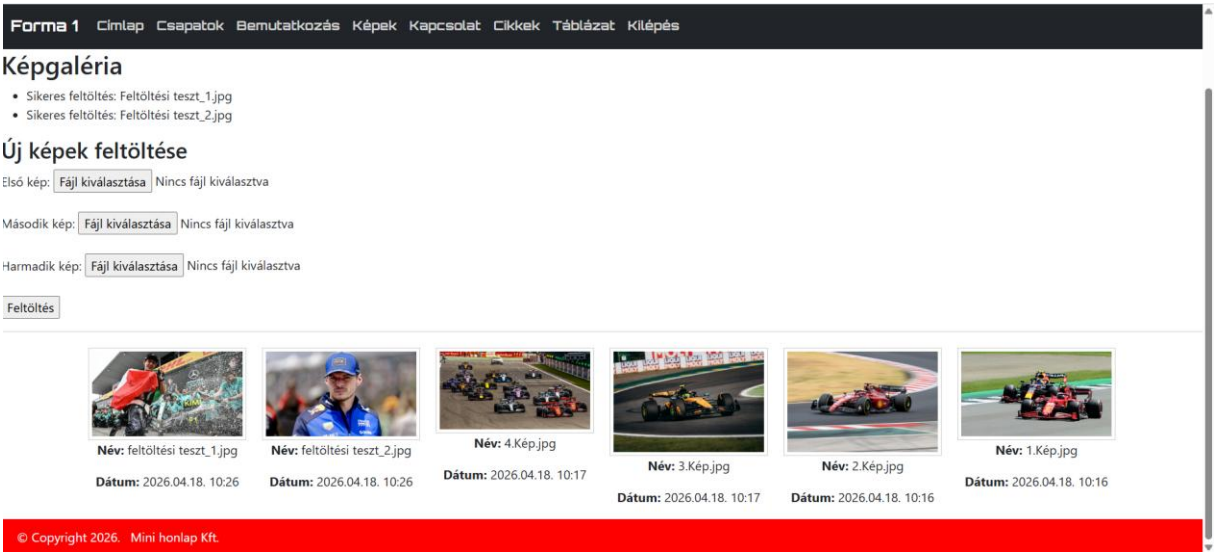


A funkció tesztelése során külön ellenőriztük a különböző felhasználói állapotokat:

- nem bejelentkezett felhasználó esetén a képgaléria megjelenik, azonban a feltöltési lehetőség nem érhető el,



- bejelentkezett felhasználó esetén a képfeltöltés sikeresen működik, és az új képek azonnal megjelennek a galériában.



A képgaléria és képfeltöltés megvalósításával sikerült egy olyan funkciót kialakítani, amely megfelel a feladat követelményeinek, biztonságosan kezeli a fájlokat, és felhasználóbarát módon jeleníti meg a képeket.

6. CRUD menü megvalósítása

A beadandó egyik legfontosabb része a CRUD menü kialakítása volt, amelynek keretében egy kiválasztott adatbázistábla rekordjainak létrehozását, megjelenítését, módosítását és törlését kellett megvalósítani útvonalkezeléssel. A feladatkiírás előírta, hogy az alkalmazás a

megadott front-controller mintára épüljön, és a CRUD funkciók a meglévő rendszerbe legyenek beillesztve, ne pedig külön, önálló projektként készüljenek el.

A választott adatbázis a **2-07-Forma-1-3 táblás** adatbázis volt, ezen belül pedig a **pilota** tábla mellett döntöttünk. A választás oka az volt, hogy ez a tábla könnyen átlátható, kevés mezőt tartalmaz, egyszerűen szerkeszthető, és nem igényli több tábla összekapcsolását. Emellett a tananyagban szereplő CRUD mintában használt users tábla szerkezete könnyen átültethető volt erre a megoldásra, így a szükséges átalakítások jól követhetően elvégezhetők voltak.

A megvalósításhoz két fő forrásmegoldást kapcsoltunk össze. Egyrészt a beadandó alapját adó **7a – PHP front-controller tervezési minta, 2. megoldás** szolgált a projekt általános szerkezetéhez, másrészt a **10a – PHP CRUD alkalmazás útvonalakkal** fejezetből vettük át a CRUD logikai felépítését. Így a feladatot nem különálló mini alkalmazásként, hanem a már meglévő beadandó struktúrájába beépítve valósítottuk meg.

A tanári mintából a következő fájlokat használtuk fel kiindulási alapként:

- db_connect.php az adatbázis-kapcsolathoz,
- createuser.php az új rekord létrehozásához,
- edituser.php a módosításhoz,
- deleteuser.php a törléshez,
- valamint a main.php listaoldali logikáját a táblázatos megjelenítéshez.

A projekt szerkezetéhez igazodva először létrehoztuk az új adatbázis-kapcsolati fájlt az includes/db_connect.php útvonalon, majd elkészítettük a három logikai fájlt a logicals mappában:

- createpilota.php
- editpilota.php
- deletepilota.php

Emellett a meglévő templates/pages/tablazat.tpl.php fájlt módosítottuk úgy, hogy ez legyen a CRUD felület központi oldala. A tanári mintából származó createuser, edituser és deleteuser fájlokat átneveztük és a saját adatbázisunkhoz igazítottuk, így jöttek létre a pilóták kezelésére szolgáló fájlok.

A pilota tábla létrehozásához külön SQL szerkezetet készítettünk, amely a következő mezőket tartalmazta:

- az – elsődleges kulcs
- nev
- nem
- szuldat
- nemzet

A CRUD működéséhez ugyanazt az adatbázist használtuk tovább, amely a bejelentkezéshez is szükséges volt, tehát nem hoztunk létre külön új adatbázist, hanem a meglévő gyakorlat7 adatbázist bővítettük egy új táblával. Ezzel egyszerűbb és egységesebb lett a projekt adatbázis-kezelése.

A db_connect.php fájl módosításakor a mintában szereplő crud adatbázisnevet a saját gyakorlat7 adatbázisunkra cseréltük, valamint kiegészítettük a kapcsolatot charset=utf8 beállítással. Ez azért volt fontos, hogy a magyar ékezetes karakterek helyesen jelenjenek meg és megfelelően kerüljenek mentésre az adatbázisban. A kapcsolatváltozó nevét is egységesítettük: a mintában szereplő $connect helyett $conn változót használtunk a további fájlokban.

A createuser.php mintát átalakítva elkészítettük a createpilota.php fájlt. Ennek során a users tábla helyére a pilota tábla került, az eredeti mezőneveket (name, email, mobile) pedig a saját mezőinkre cseréltük (nev, nem, szuldat, nemzet). Ugyanezt a logikát követtük az editpilota.php és deletepilota.php fájloknál is. A módosítás során figyelnünk kellett arra is, hogy az adatbázisban az elsődleges kulcs mező neve nem id, hanem az, ugyanakkor az URL-ben továbbra is az id GET paramétert használtuk, mert ez egyszerűbbé tette a minta logikájának megtartását.

A CRUD listaoldal kialakításakor a tanári mintában található main.php szerkezetét ültettük át a tablazat.tpl.php fájlba. Meghagytuk a táblázatos szerkezetet, a while ($data = $stmt->fetch(...)) ciklust, valamint a Bootstrap-alapú megjelenést is, mivel ez jól illeszkedett a feladatban megadott mintához. A users helyett itt is a pilota tábla rekordjait kérdeztük le, és az oszlopokat a következő mezőkhöz igazítottuk:

- az

- nev
- nem
- szuldat
- nemzet

A listaoldalon elhelyeztünk egy **Új pilóta** gombot, valamint minden rekordhoz külön **Módosítás** és **Törlés** gombot. A hivatkozások már nem közvetlenül fájlnevekre mutattak, hanem a front-controller logikájához igazodva GET paraméterekkel működtek, például:

- index.php?oldal=tablazat&action=create
- index.php?oldal=tablazat&action=edit&id=...
- index.php?oldal=tablazat&action=delete&id=...

Ez azért volt szükséges, mert a projektünk minden oldalt az index.php központi belépési ponton keresztül kezel.

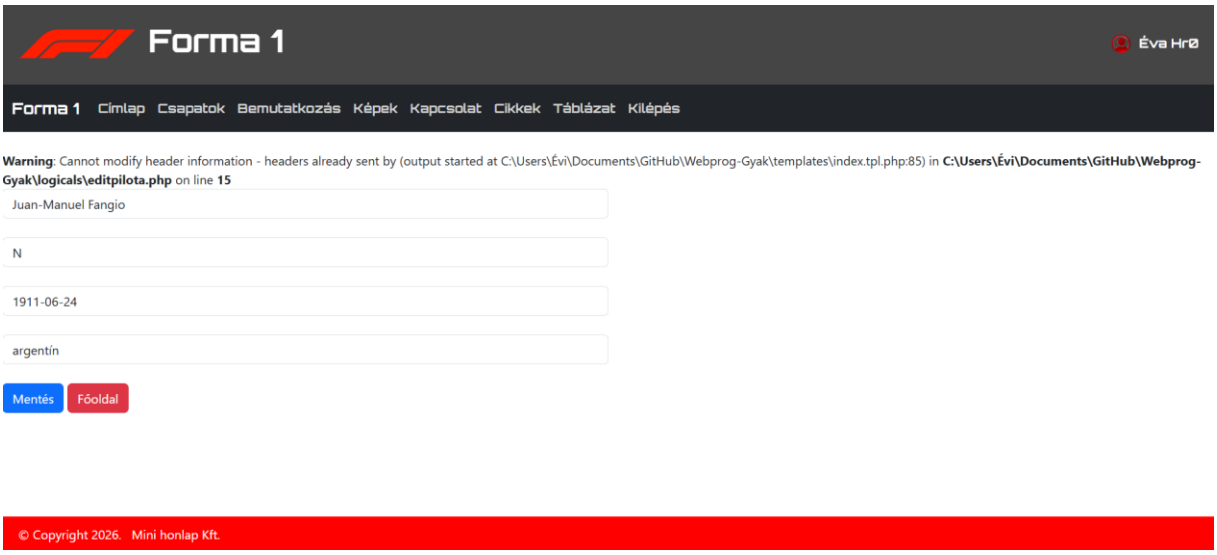
A CRUD rendszer működéséhez a tablazat.tpl.php tetején bevezettük az action alapú vezérlést. Ennek segítségével a rendszer a GET paraméter alapján eldönti, hogy a listaoldalt, a létrehozó űrlapot, a módosító űrlapot vagy a törlési műveletet hajtsa végre. Ez a megoldás tette teljessé az útvonalkezelést, és biztosította, hogy a CRUD menü egyetlen menüponton belül, mégis jól elkülönített funkciókkal működjön.

A fejlesztés során több hibával is találkoztunk. Az egyik legfontosabb probléma az volt, hogy a logikai fájlokban kezdetben rossz útvonalakat használtunk az includes/db_connect.php eléréséhez, mivel a mintában szereplő relatív útvonalak nem illeszkedtek a saját projektünk felépítéséhez. Ezt az útvonalak javításával oldottuk meg. Egy másik fontos hibát a header("Location: ...") sorok okoztak. Ezek a PHP átirányítások a mi projektünkben már a HTML kimenet megkezdése után futottak le, ezért Cannot modify header information hibát eredményeztek.

A probléma megoldására a szerveroldali átirányítást JavaScriptes átirányításra cseréltük:

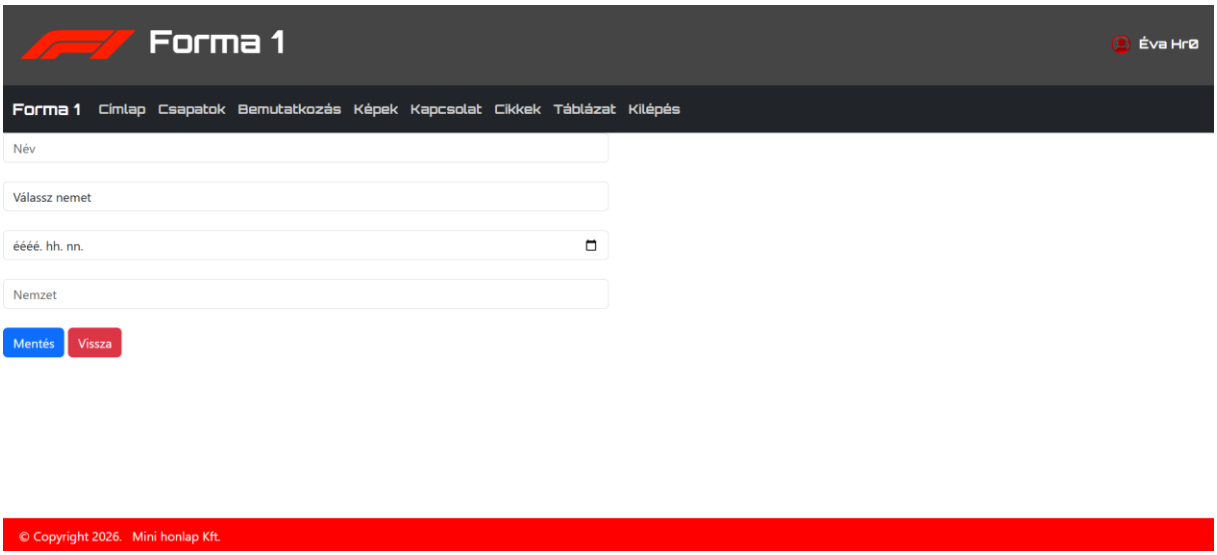
```
echo "<script>window.location.href='index.php?oldal=tablazat';</script>";
exit;
```

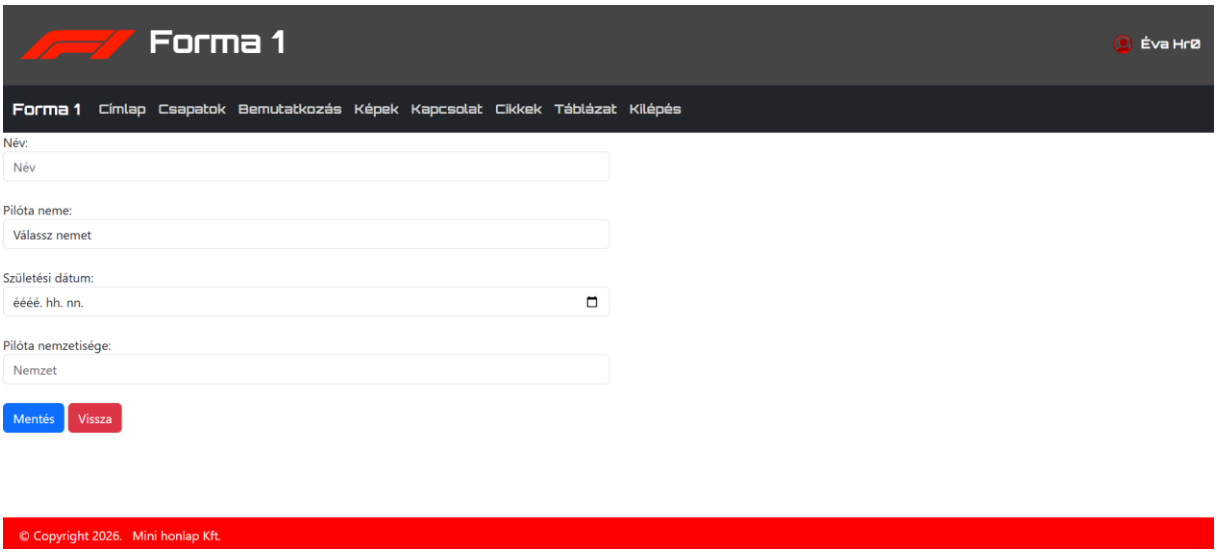
Ezzel a módosítással a mentés, módosítás és törlés után a rendszer hibamentesen vissza tudott térni a táblázatos listaoldalra.



A CRUD űrlapok használhatóságát a fejlesztés végén tovább javítottuk. A nem mező eredetileg szabad szövegmező volt, azonban ez nem volt ideális, mert a felhasználó tetszőleges értéket írhatott be. Ezt ezért legördülő listára cseréltük, ahol csak az F és N értékek választhatók. Ugyanezt a módosítást az edit űrlapon is elvégeztük, ahol az aktuális érték automatikusan kiválasztva jelenik meg. Ezzel csökkentettük a hibás adatbevitel lehetőségét.

A szuldat mezőt szintén továbbfejlesztettük. A kezdeti szabad szövegmező helyett type="date" típusú mezőt alkalmaztunk, így a dátumok egységes formátumban adhatók meg. Mivel a böngésző alapértelmezett megjelenítése nem írta ki egyértelműen a mező nevét, a végleges megoldásban külön label elemeket helyeztünk el a mezők fölé. Ez nemcsak a dátum mezőnél, hanem végül az összes beviteli mezőnél egységesebb, átláthatóbb megjelenést eredményezett.





A CRUD menü megvalósítása során tehát sikerült egy teljes értékű, útvonalkezeléssel működő alkalmazást kialakítani, amely megfelel a beadandó követelményeinek. A rendszer képes új rekordok létrehozására, a meglévő adatok megjelenítésére, módosítására és törlésére, miközben megjelenésében is követi a feladatban megadott mintát. A fejlesztés során felmerült hibák javítása, valamint az űrlapok továbbfejlesztése hozzájárult ahhoz, hogy a végeredmény ne csak technikailag működőképes, hanem felhasználói szempontból is átlátható és jól használható legyen.

7. Főoldal megvalósítása

A főoldal kialakítása során fontos szempont volt, hogy az oldal látványos, jól áttekinthető és reszponzív legyen, valamint megfeleljen a feladatkiírásban szereplő tartalmi elvárásoknak. A megvalósítás során a Bootstrap keretrendszert használtuk, amely nagy segítséget nyújtott abban, hogy az oldal különböző képernyőméreteken is megfelelően jelenjen meg.

A tartalmi rész elrendezését kétoszlopos szerkezetben alakítottuk ki. Ez a megoldás nagyobb kijelzőn áttekinthető és esztétikus megjelenést biztosít, kisebb képernyőméret esetén pedig az elemek automatikusan egymás alá rendeződnek. Ennek köszönhetően a főoldal mobiltelefonon és tableten is jól használható marad.

A feladat követelményeinek megfelelően a főoldalon két különböző típusú videót is elhelyeztünk. Az egyik egy beágyazott YouTube-videó, amely külső szolgáltatótól származik, a másik pedig egy saját könyvtárból betöltött, MP4 formátumú videó, amelynek hossza nem

haladja meg az 5 másodpercet. A videók beágyazását HTML5 elemek segítségével valósítottuk meg, így ezek egyszerűen és szabványos módon kerültek be az oldalba.

A főoldalon továbbá egy Google Térkép is helyet kapott, amely a választott téma szempontjából releváns helyszínt jeleníti meg. Mivel a projekt témája a Forma–1-hez kapcsolódik, ezért a magyarországi Forma–1-es versenyek helyszínét, a Hungaroringet ágyaztuk be az oldalra. Ezzel teljesítettük a feladat azon részét is, amely szerint a főoldalon térképes beágyazásnak is szerepelnie kell.

A főoldal kialakítása során a navigációs menüt is finom hangoltuk. Pontosítottuk a menüpontok megnevezését, módosítottuk azok sorrendjét, és eltávolítottuk azokat az elemeket, amelyek nem voltak szükségesek a beadandó követelményeinek teljesítéséhez. Így a végleges menüszerkezet már teljes mértékben igazodott a kiírásban szereplő funkciókhoz, és áttekinthetőbbé vált a felhasználók számára.

Összességében a főoldal kialakítása során sikerült egy olyan kezdőfelületet létrehozni, amely vizuálisan rendezett, technikailag megfelelően működik, reszponzív kialakítású, és teljesíti a feladatban előírt multimédiás és térképes követelményeket.

8. Kapcsolat menü megvalósítása

A projekt egyik részeként létrehoztuk a Kapcsolat menüpontot, amely egy egyszerű űrlapot tartalmaz a felhasználók számára. Az űrlap célja, hogy a látogatók üzenetet küldhessenek az oldal tulajdonosának, így teljesítve a feladatkiírásban szereplő követelményeket.

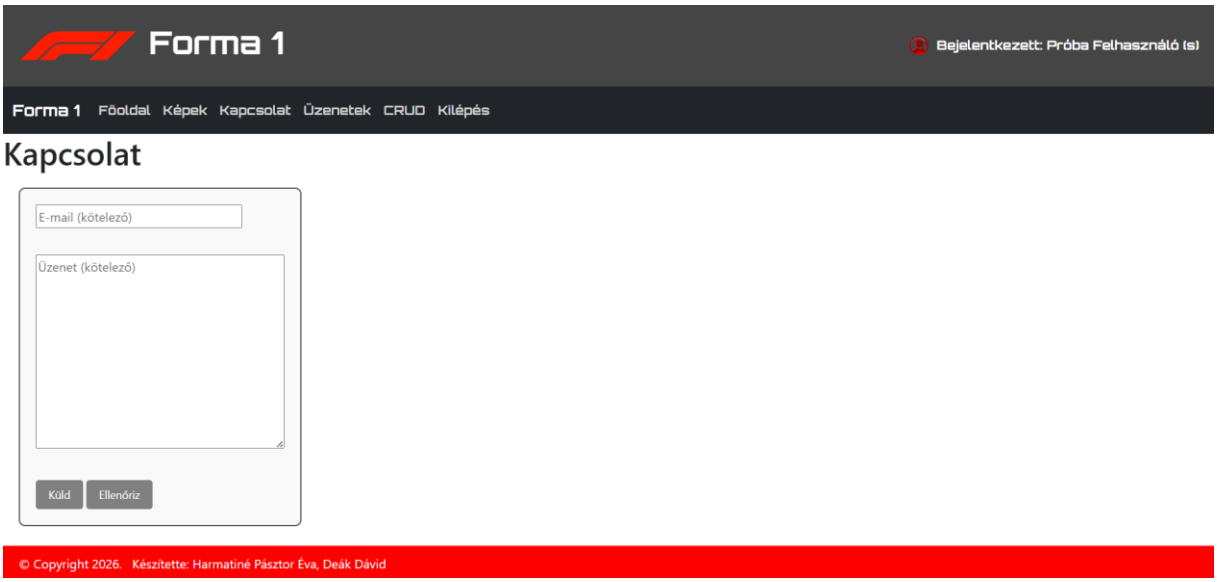
Az űrlap két beviteli mezőt tartalmaz, valamint két gombot: egy **„Küld”** és egy **„Ellenőriz”** gombot. A „Küld” gomb az adatok elküldéséért felelős, míg az „Ellenőriz” gomb a kliensoldali ellenőrzést indítja el. Az űrlap megjelenését CSS segítségével formáztuk, amely során keretet, lekerekített sarkokat, valamint megfelelő margókat alkalmaztunk. Ennek eredményeként egy esztétikus és jól elkülönülő felhasználói felület jött létre.

A kliensoldali ellenőrzést JavaScript segítségével valósítottuk meg. Ehhez az órai anyagban szereplő mintakódot vettük alapul, amelyet a projekt igényeihez igazítottunk. A kódból eltávolítottuk a felesleges részeket, és csak azokat az ellenőrzéseket hagytuk meg, amelyek az űrlap helyes kitöltésének vizsgálatához szükségesek. A végleges JavaScript fájlt **„ellenorzes.js”** néven hoztuk létre, amely a projekt logikai fájljai között („logicals” mappa) kapott helyet.

A szerveroldali ellenőrzést PHP segítségével valósítottuk meg, amely biztosítja, hogy az adatok akkor is ellenőrzésre kerüljenek, ha a kliensoldali JavaScript valamilyen okból nem fut le. Ez a kettős ellenőrzés növeli az alkalmazás megbízhatóságát és biztonságát.

A szerveroldali megvalósítás során egy hibába ütköztünk a session_start(); függvény használatakor. A probléma abból adódott, hogy a session kezelés nem minden esetben indult el megfelelően. Ezt a hibát úgy oldottuk meg, hogy a session_start(); hívást egy olyan feltételbe helyeztük, amely először ellenőrzi, hogy a session már elindult-e. Így elkerültük a többszöri inicializálásból adódó hibákat, és biztosítottuk a stabil működést.

A kapcsolat menü megvalósításával sikerült egy olyan funkciót létrehozni, amely megfelel a feladat követelményeinek, biztosítja az adatok megfelelő ellenőrzését mind kliens-, mind szerveroldalon, valamint felhasználóbarát módon teszi lehetővé az üzenetküldést.



9. Üzenetek menü megvalósítása

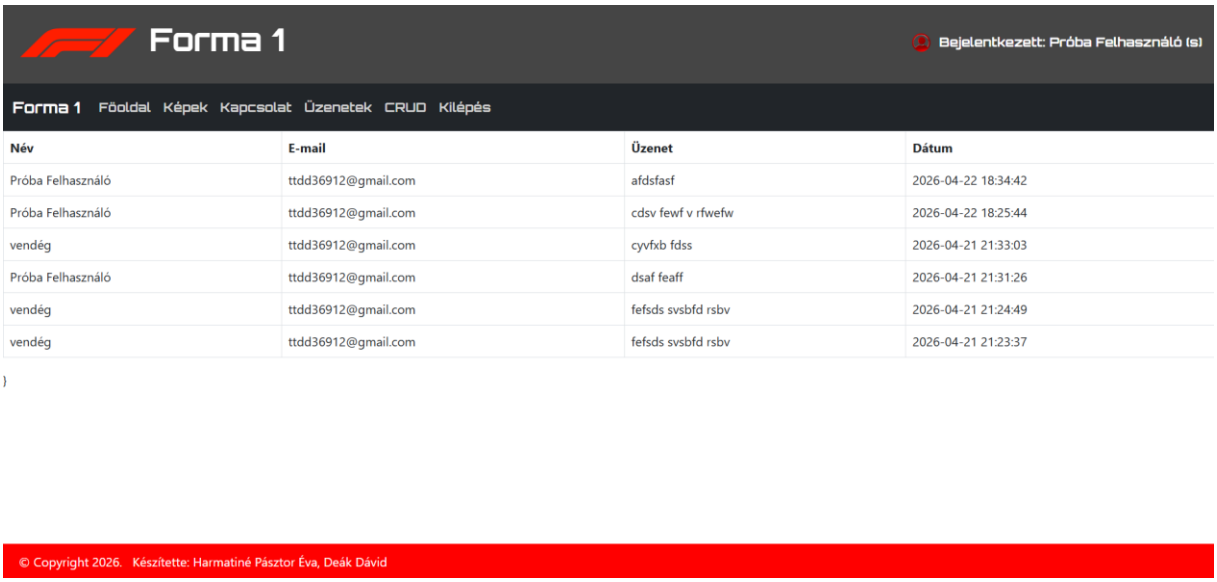
A kapcsolat űrlaphoz kapcsolódóan létrehoztuk az Üzenetek menüpontot, amely lehetővé teszi az elküldött üzenetek megtekintését az adatbázisból. A feladat követelményeinek megfelelően ezt a menüpontot úgy alakítottuk ki, hogy kizárólag bejelentkezett felhasználók számára legyen elérhető.

A jogosultságkezelést a config.inc.php fájlban oldottuk meg, ahol egy új menüpontot hoztunk létre az alábbi beállítással: 'menun' => array(0, 1). Ez a konfiguráció biztosítja, hogy a

menüpont csak akkor jelenjen meg, ha a felhasználó be van jelentkezve, így megfelel a feladat biztonsági követelményeinek.

Az üzenetek megjelenítésére egy egyszerű, táblázatos elrendezést választottunk. Ez a megoldás jól áttekinthetővé teszi az adatokat, és lehetővé teszi, hogy a felhasználó gyorsan átlássa az elküldött üzeneteket. A táblázat oszlopai tartalmazzák az üzenet küldőjének nevét, az üzenet tartalmát, valamint a küldés időpontját.

A megvalósítás során figyelembe vettük azt a követelményt is, hogy ha az üzenetet nem bejelentkezett felhasználó küldte, akkor a rendszer „Vendég” néven jelenítse meg a küldőt. A funkció tesztelése során több tesztüzenetet is rögzítettünk az adatbázisban, és ellenőriztük azok helyes megjelenítését. A tesztek során megállapítottuk, hogy az üzenetek megfelelően kerülnek mentésre és visszaolvasásra, valamint a megjelenítés megfelel az elvárt formátumnak.



Név	E-mail	Üzenet	Dátum
Próba Felhasználó	ttdd36912@gmail.com	afdsfasf	2026-04-22 18:34:42
Próba Felhasználó	ttdd36912@gmail.com	cdsv fewf v rfwefw	2026-04-22 18:25:44
vendég	ttdd36912@gmail.com	cyvfxb fdss	2026-04-21 21:33:03
Próba Felhasználó	ttdd36912@gmail.com	dsaf feaff	2026-04-21 21:31:26
vendég	ttdd36912@gmail.com	fefsds svsbfd rsbv	2026-04-21 21:24:49
vendég	ttdd36912@gmail.com	fefsds svsbfd rsbv	2026-04-21 21:23:37

Az Üzenetek menüpont megvalósításával sikerült egy olyan funkciót kialakítani, amely szorosan kapcsolódik a kapcsolat űrlaphoz, biztosítja az adatok strukturált megjelenítését, valamint megfelel a jogosultsági és adatkezelési követelményeknek.

10.Rövid használati útmutató a honlaphoz

A webalkalmazás használata a felső navigációs menüsoron keresztül történik, amely lehetővé teszi az egyes funkciók gyors és egyszerű elérését. A menüpontok megjelenése a felhasználó bejelentkezési állapotától függ, így a rendszer dinamikusan alkalmazkodik a felhasználói jogosultságokhoz.

Amennyiben a felhasználó nincs bejelentkezve, a menüsor a

- Főoldal,
- Képek,
- Kapcsolat,
- CRUD és
- Belépés

menüpontokat tartalmazza. A Főoldal alatt általános információk találhatók a Forma–1 világáról, valamint különböző multimédiás elemek segítik a tartalom szemléltetését. A Képek menüpontban a látogatók megtekinthetik a feltöltött képeket, azonban új képek feltöltésére ebben az állapotban még nincs lehetőségük. A Kapcsolat oldalon keresztül minden felhasználó üzenetet küldhet, amelyet a rendszer eltárol az adatbázisban. A CRUD menüpont a pilóták adatainak kezelésére szolgál, ahol a felhasználók megtekinthetik és módosíthatják az adatokat. A Belépés menüpont biztosítja a rendszerbe való bejelentkezés lehetőségét.

Sikeres bejelentkezést követően a menüsor kibővül, és megjelenik az

- Üzenetek és a
- Kilépés

menüpont is. A Képek menüpont ebben az esetben már lehetőséget biztosít új képek feltöltésére is, így a felhasználók aktívan hozzájárulhatnak a galéria bővítéséhez. Az Üzenetek menüpont kizárólag bejelentkezett felhasználók számára érhető el, és az adatbázisban tárolt üzenetek áttekintését teszi lehetővé. A felület jobb felső sarkában megjelenő név és ikon folyamatos visszajelzést ad a felhasználó aktuális bejelentkezési állapotáról. A Kilépés funkció segítségével a felhasználó bármikor kijelentkezhet a rendszerből, amely után a jogosultsághoz kötött funkciók ismét korlátozásra kerülnek.

Összességében a webalkalmazás logikus felépítésű, a funkciók jól elkülönülnek egymástól, így a használata könnyen elsajátítható és áttekinthető mind bejelentkezett, mind anonim felhasználók számára.

11.Összegzés

A webalkalmazás elkészítése során a kitűzött célunk az volt, hogy a beadandó feladat követelményeit ne csupán technikai szempontból teljesítsük, hanem egy átgondolt, egységes

megjelenésű és felhasználóbarát rendszert hozzunk létre. A fejlesztés eredményeként minden előírt funkciót sikeresen megvalósítottunk, beleértve a front-controller alapú működést, a felhasználókezelést, a képfeltöltést, valamint a CRUD alkalmazást is.

A projektet kétfős csapatban készítettük el (Harmatiné Pásztor Éva – JBDAQO és Deák Dávid – OPFHRV). Míg a korábbi, előadás beadandó feladat során a fejlesztés több esetben személyes konzultáció keretében zajlott, ennél a projektnél már teljes mértékben online együttműködésben dolgoztunk. Az előző beadandó során szerzett tapasztalatok nagyban megkönnyítették a közös munkát, mivel már kialakult egy hatékony munkamódszerünk, és tisztában voltunk azzal, hogyan tudjuk a feladatokat megfelelően felosztani és összehangolni.

A fejlesztési folyamat során jelentős segítséget nyújtott a tananyaghoz biztosított **megoldasok.zip** fájl, amely stabil alapot adott az egyes funkciók kialakításához. Ezeket a mintákat nem változtatás nélkül vettük át, hanem minden esetben a saját projektünk igényeihez igazítottuk, mind működés, mind megjelenés szempontjából. Ennek köszönhetően a végeredmény egy egyedi, a témához illeszkedő webalkalmazás lett.

Az adatbázis kiválasztása szintén tudatos döntés volt. A „2-07-Forma-1-3 táblás” adatbázis mellett azért döntöttünk, mert jól átlátható szerkezetű, könnyen kezelhető, és tartalmilag is alkalmas egy látványos weboldal kialakítására. Emellett fontos szempont volt az is, hogy ehhez a témához könnyen találtunk megfelelő multimédiás tartalmakat, amelyek segítették az oldal vizuális megjelenésének kialakítását.

A projekt során a GitHub használata továbbra is kiemelt szerepet kapott, amely lehetővé tette a verziókövetést és a fejlesztési folyamat átlátható dokumentálását. A commitok (Harmatiné Pásztor Éva – harmatinepasztoreva és Deák Dávid – ttdd33) segítségével jól követhetővé vált a munka előrehaladása, hasonlóan az előző beadandó dokumentációban bemutatott módszerhez.

Összességében elmondható, hogy a projekt elkészítése során nemcsak a beadandó követelményeket sikerült teljesítenünk, hanem értékes gyakorlati tapasztalatokat is szereztünk a webfejlesztés, az adatbázis-kezelés és az online csapatmunka területén. A korábbi tapasztalatokra építve hatékonyabban és tudatosabban tudtuk megvalósítani ezt a feladatot, amely egy komplex, jól működő webalkalmazás létrehozásához vezetett.

12.Források

A projekt elkészítése során a következő forrásokat és segédanyagokat használtuk fel:

- Web-programozas-1-gyakorlat PDF
- A tanár úr által biztosított megoldasok.zip fájl
- A megadott adatbázis-forrásokat tartalmazó Google Drive mappa: „2-07-Forma-1-3 táblás”
- GitHub repository: https://github.com/ttdd33/Webprog-Gyak